Research and Development of an Automated System for Concept Drift Detection and Adaptive Machine Learning Model Updating

Master's Thesis

Student: Le Phuc Duc
Supervisor 1: Assoc.Prof.Dr. Thoai Nam
Supervisor 2: Dr. Nguyen Quang Hung

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

2026

Presentation Outline

Agenda

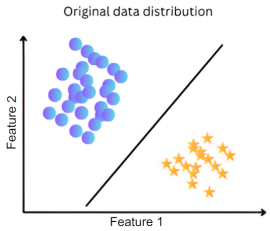
The Core Problem: Concept Drift

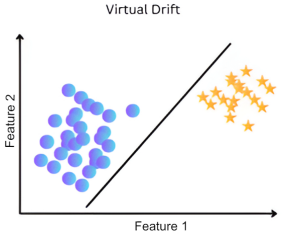
Mathematical Definition

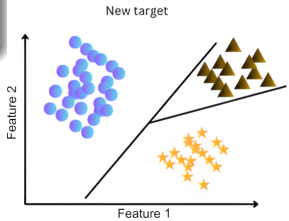
$$P_{\text{train}}(X, Y) \neq P_{\text{online},t}(X, Y)$$
$$P(X, Y) = P(X)\,P(Y \mid X)$$

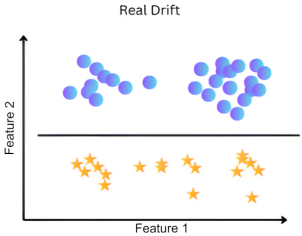
Real-world Impact:

- Banking Fraud (False Alarms ↑)
- Predictive Maintenance (Accuracy ↓)





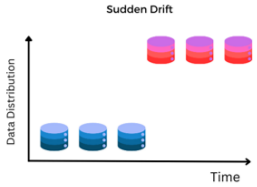


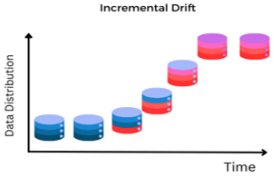


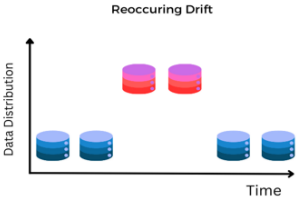
Concept Drift: Distribution vs Pattern

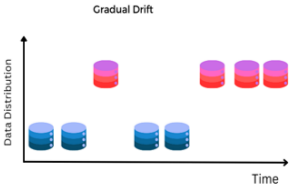
Virtual Drift:
Only $P(X)$ changes.

Real Drift:
Decision boundary $P(Y|X)$ changes.









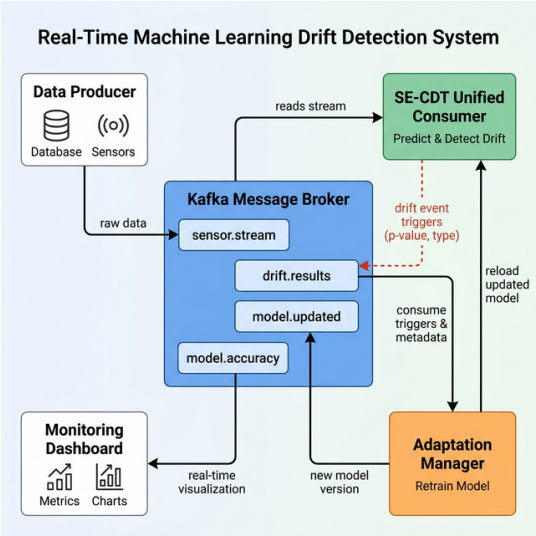
Objectives & System Architecture

A three-layer pipeline:

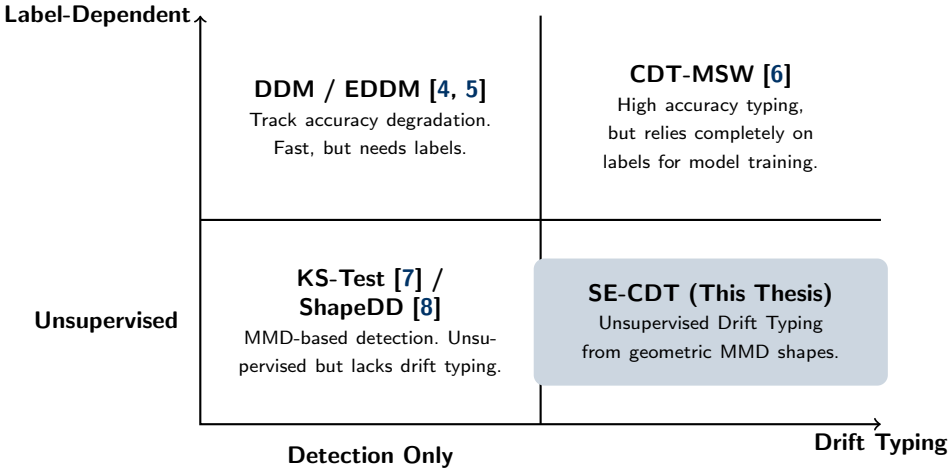
- **L1 — Detect** *(When?)*
  ShapeDD-IDW: fast, unsupervised, $\alpha = 0.05$ calibrated.
- **L2 — Classify** *(What?)*
  SE-CDT: label-free shape analyzer, 5 drift types.
- **L3 — Adapt** *(How?)*
  Type-aware router: reset / warm-start / reuse.

Goals

Fast, reliable detection; unsupervised drift typing (no labels); end-to-end validation.



Related Works: The Gap in Current Literature



Evaluated Datasets: SEA, RTG, RBF, Sine [1, 2, 3]

Agenda

MMD: The Unsupervised Drift Signal

Why unsupervised?

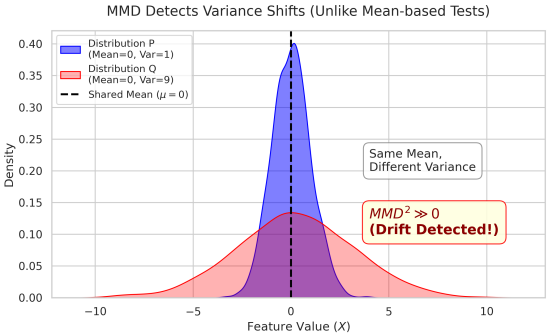
- Supervised detection needs ground-truth labels right after each prediction.
- In real application, labels are delayed or expensive → monitor the **raw distribution** $P(X)$ instead.

Maximum Mean Discrepancy: map points into an RKHS and compare their mean embeddings ("centers of mass").

MMD Statistic

$$\widehat{\mathrm{MMD}}^2 = \underbrace{\text{Within } P}_{\text{self-sim}} + \underbrace{\text{Within } Q}_{\text{self-sim}} - \underbrace{\text{Cross } P \leftrightarrow Q}_{\text{dis-sim}}$$

$\widehat{\mathrm{MMD}}^2 \approx 0$: same distribution; $\widehat{\mathrm{MMD}}^2 \gg 0$: **drift candidate**.



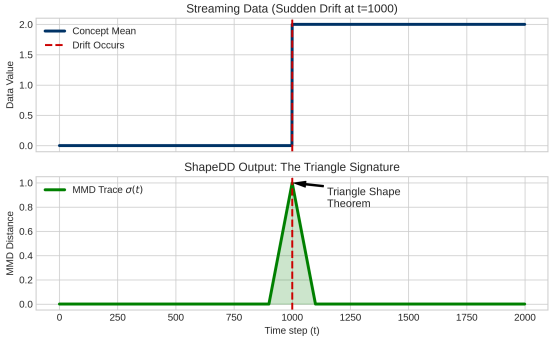
From MMD Trace to ShapeDD's Triangle

Stream → trace:

- A sliding window of size $2l_1$ splits into W_{ref} and W_{test}; their MMD at each step forms a continuous trace $\sigma(t)$.
- Stable data → small noise floor; crossing a drift → $\sigma(t)$ rises into a peak.

ShapeDD's insight — the triangle:

- A sudden drift yields a **triangular** $\sigma(t)$ (linear contamination as the window crosses the drift) — detect that *shape*, not a raw threshold.
- Peak height $\sigma(t_c) =$ **drift magnitude** $\approx \text{MMD}^2(P, Q)$.



Bottleneck

Significance via a **permutation test** rebuilds the kernel thousands of times. We replace it with **IDW weighting** + **Gamma-null** (next).

Agenda

Inverse Density-Weighted MMD (IDW-MMD)

Uniform weights ($1/n^2$):

- Dense majority clusters dominate the statistic.
- Early drifts at distribution **boundaries** get washed out.

IDW weighting (the idea)

Weight each point by **inverse local density**:

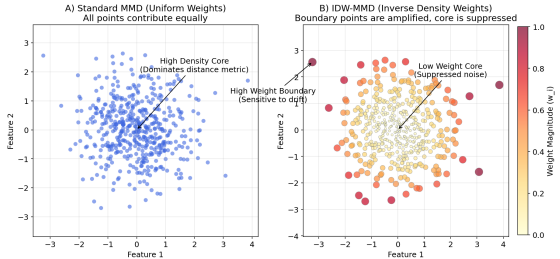
$$w_i \propto \frac{1}{\sqrt{d_i}+\epsilon}, \qquad d_i = \sum_j k(x_i, x_j)$$

Dense core $\rightarrow$ small w_i; sparse **boundary** $\rightarrow$ large w_i.

Full IDW-MMD Statistic

$$\widehat{\mathrm{MMD}}^2_{\mathrm{IDW}} = \underbrace{\sum_{i\neq j} W^{XX}_{ij}\, k(x_i, x_j)}_{\text{Within } P \text{ (Weighted)}} + \underbrace{\sum_{p\neq q} W^{YY}_{pq}\, k(y_p, y_q)}_{\text{Within } Q \text{ (Weighted)}} - 2 \underbrace{\sum_{i,p} \frac{1}{nm} k(x_i, y_p)}_{\text{Cross } P\leftrightarrow Q \text{ (Uniform)}}$$

- **Inverse Density** (W^{XX}_{ij}, W^{YY}_{pq}): amplifies boundary points.
- **Uniform weight** (XY term): anchors geometric overlap.



Result: sensitive to early boundary changes, no extra false alarms on the bulk.

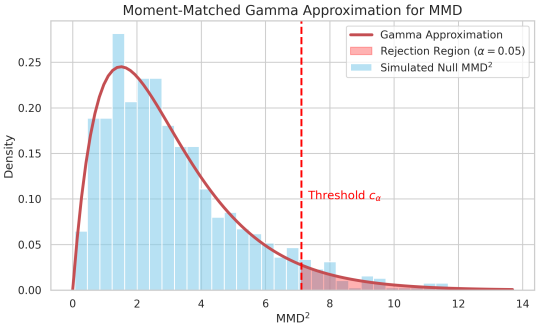
Statistical Calibration (Gamma-Null Approximation)

The Permutation Bottleneck:

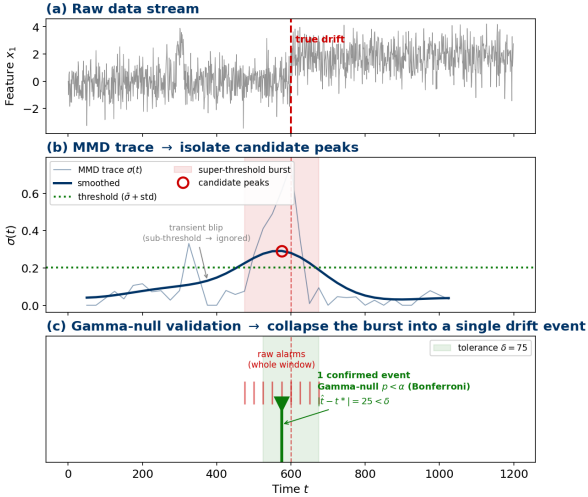
- Standard ShapeDD runs $B = 2500$ permutations to estimate p-value.
- Each permutation re-builds the kernel matrix $\rightarrow$ extremely slow.

Gamma-Null Approximation Solution:

- Under H_0, the MMD statistic is approximately Gamma-distributed.
- We estimate the first two moments (mean & variance) from a small number of bootstrap shuffles ($B = 20$), reusing the kernel matrix.
- Matches accuracy while achieving a **7–10× end-to-end speedup**.



ShapeDD-IDW in Action: MMD Trace → Single Drift Event



The detector, end-to-end:

1. **(a)** Stream → compute the **IDW-MMD trace** $\sigma(t)$.
2. **(b) Isolate peaks:** smooth $\sigma(t)$, keep points above $\bar{\sigma} + \mathrm{std}$. **Ignore blips under threshold**.
3. **(c) Validate & group:** the Gamma-null p-value (Bonferroni-adjusted) confirms the peak; the whole super-threshold **burst collapses to one event** within tolerance δ.

Outcome

One calibrated detection per true drift — no permutation test, few false alarms.

Agenda

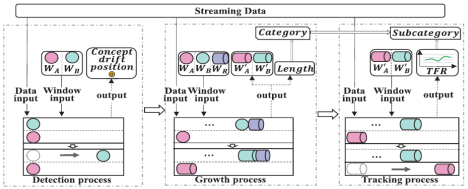
CDT-MSW — Supervised Classification Baseline

CDT-MSW (Guo et al., 2022) groups drift into:

- **TCD (Transient):** Sudden, Blip, Recurrent.
- **PCD (Progressive):** Gradual, Incremental.

Growth Process:

- Tracks the variance of accuracy degradation over a multi-sliding window.
- Short degradation $\rightarrow$ TCD; sustained degradation $\rightarrow$ PCD.



Limitation

CDT-MSW needs **ground-truth labels** at every step to compute the accuracy curve — impractical for data streams where labels arrive late or never.

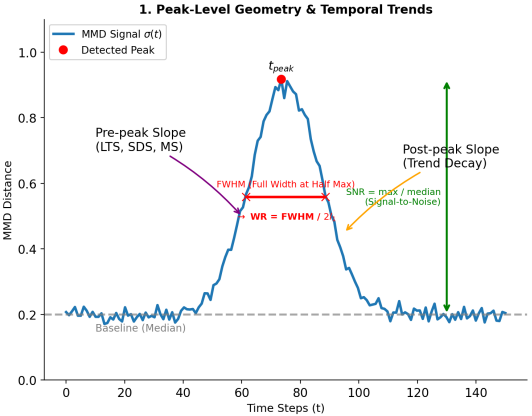
Motivation for SE-CDT

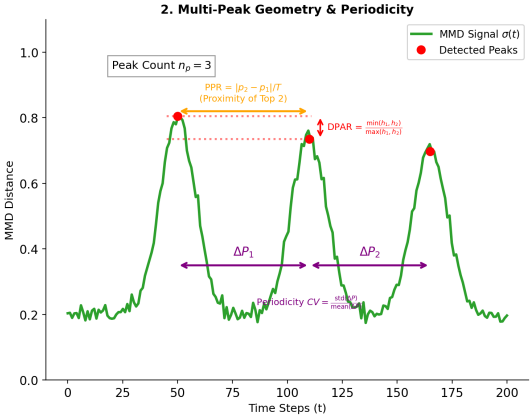
Replace the accuracy curve with the *shape* of the MMD signal $\sigma(t)$. The same TCD/PCD distinction can then be made **without any labels at runtime**.

SE-CDT: Reading the MMD Shape (9 Attributes)

Type drift without labels by reading the shape of $\sigma(t)$ at the detection point:

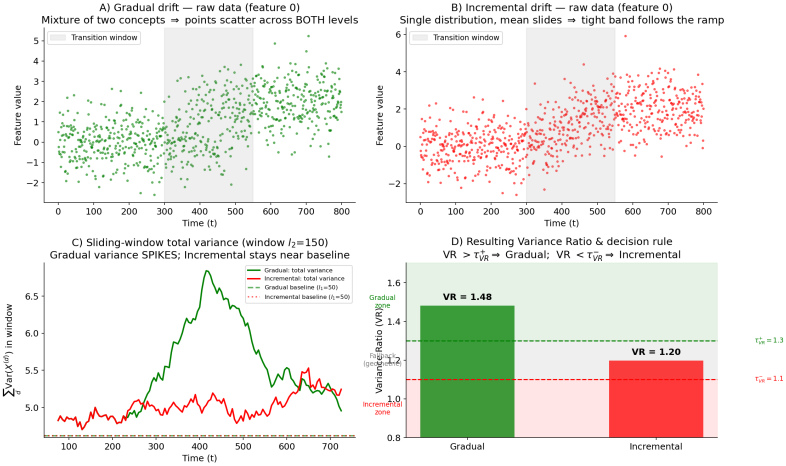
- **Geometric:** n_p (peak count), WR (width ratio), CV (coeff. of variation), SNR, PPR, DPAR.
- **Temporal:** LTS (linear trend), SDS (step score), MS (monotonicity).
- + **VR:** one *raw-data* signal (not from $\sigma(t)$); the slow-drift tie-breaker.





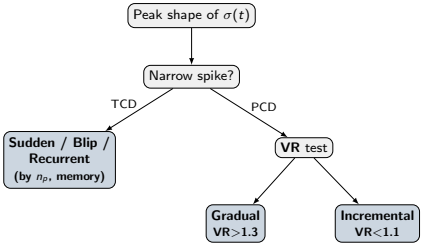
SE-CDT: Variance Ratio (VR) for Slow Drifts

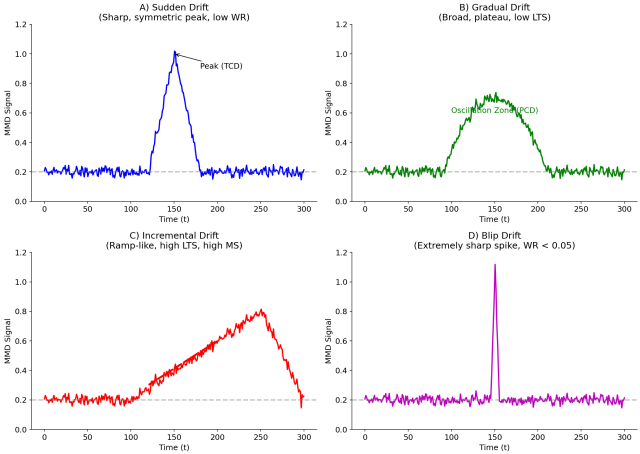
Challenge: Gradual and Incremental share flat MMD humps.
Solution: Variance Ratio (VR). Gradual (mixture) ⇒ variance **spike**. Incremental (single shift) ⇒ **flat**.



SE-CDT: Decision Logic & Drift Signatures

Simplified decision tree:





Hybrid Pipeline: Standard MMD to Trace, IDW-MMD to Validate

Why use two MMD variants instead of one?

- **Standard MMD:** Fast, preserves the natural geometry of the trace (gradual rising trends, duration) needed for downstream shape classification.
- **IDW-MMD:** Highly sensitive to shift, rigorously controls Type-I error ($\alpha = 0.05$). Density computation is costly, so it's only used for validation.

Two-Stage Architecture

- **Stage 1 (Trace):** Run Standard MMD continuously to find candidate peaks.
- **Stage 2 (Validate):** Compute IDW-MMD (p-val) only at peaks. If confirmed, pass the Standard MMD trace shape to the SE-CDT Classifier.

Agenda

Optimizing Update Cost

Detected Drift Type	Adaptation Policy
Sudden	**Full Reset**
Incremental	**Continuous Update**
Gradual	**Weighted Window**
Recurrent	**Model Reuse (from Memory)**
Blip	**Ignore (Do Nothing)**

Goal

Minimize catastrophic forgetting and compute overhead by avoiding a "one-size-fits-all" retraining policy.

Kafka Integration

Triggers downstream microservices to hot-swap or fine-tune models dynamically.

Agenda

Datasets & Experimental Setup

14 Evaluation Datasets (Grouped):

- **Sudden** $P(X)$**:** Gaussian Shift (Moderate), GCS (3 concepts), 4 $\times$ Random Uniform (Sensitivity tests: Mild to Ultra Severe).
- **Blip** $P(X)$**:** RBF Blips (short-term recurring centroid shifts).
- **Real Drift** $P(Y|X)$ **(Control Group):** Standard SEA, Rotating Hyperplane, LED Abrupt (no feature drift; to verify negative controls).
- **Semi-Synthetic:** Electricity Semi-Synthetic (real features, 10 injected drift points).
- **Slow & Cycle:** Gaussian Gradual (linear transition), GRS (Recurrent $A \leftrightarrow B$), Gaussian Stationary (H0 calibration).

Experimental Protocol

- **Runs:** 30 independent seeds per stream to ensure statistical stability.
- **Layer 1 (Detection):** Tolerance $\delta = 75$, cooldown $= 150$. Measured via F1-score, EDR, and False Positives.
- **Layer 2 (Classification):** Oracle Mode (1800 events total: 6 scenarios $\times$ 30 seeds $\times$ 10 events) to measure CAT and SUB accuracy.
- **Layer 3 (Adaptation):** Prequential accuracy (test-then-train) on Stepping and Sudden streams.

Classification Setup in Oracle Mode

Streams (5-D Gaussian, 1800 events total):

Scenario	Composition
Mixed A	Sudden → Gradual → Recurrent
Mixed B	Blip → Incremental → Sudden
Repeated Sudden	10 × Sudden
Repeated Gradual	10 × Gradual
Repeated Incremental	10 × Incremental
Repeated Recurrent	10 × Recurrent

6 scenarios × 30 seeds × 10 events = **1800 events**. Each non-recurrent event shifts a different feature subset, so new concepts are genuinely distinct.

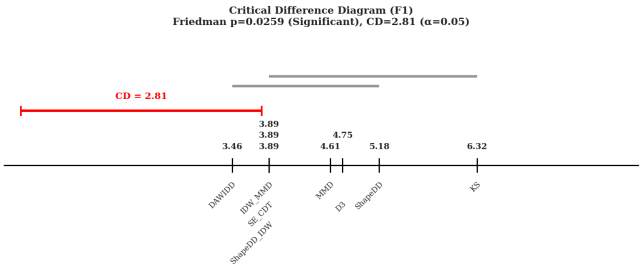
Oracle mode

The classifier receives the ground-truth drift *position* and is scored only on the *typing* step. This isolates classification skill from detection skill — the same way the detection protocol ($\delta = 75$, cooldown) isolates detection.

Two metrics

CAT — TCD vs PCD (the decision that drives adaptation).
SUB — exact 5-class subtype (micro / macro).

Result 1a: Drift Detection Performance



Statistical Significance (Nemenyi Test):

- **Meaning:** Post-hoc pairwise comparison of detectors across all 14 datasets.
- **Critical Difference (CD):** Threshold of 2.81 ($\alpha = 0.05$).
- **Interpretation:** Groups connected by a thick line (like DAWIDD, MMD, and SE-CDT) have **no statistically significant difference** in performance.
- **Result:** SE-CDT (rank 3.89) is statistically tied with DAWIDD (rank 3.46).

Result 1b: Type-I Error on Stationary Data

Type-I Error at nominal $\alpha = 0.05$

Distribution	Standard MMD (perm)	IDW-MMD (Gamma null)	SE-CDT (composite)
Gauss-iid d=5	0.025 ± 0.022	0.040 ± 0.027	0.115 ± 0.044
Gauss-iid d=10	0.055 ± 0.032	0.050 ± 0.030	0.080 ± 0.038
Gauss-AR1 d=5	0.045 ± 0.029	0.075 ± 0.037	0.140 ± 0.048

Evaluation on Stationary Streams:

- Evaluated across 3 stationary datasets (no drift) over 30 runs to measure false alarms.
- **IDW-MMD Calibration:** Holds extremely close to the nominal rate ($\alpha = 0.05$), ranging between 0.025–0.075.
- **Composite Trace (SE-CDT):** Becomes anti-conservative (0.080–0.140) due to peak selection and sliding-window dependency.

Result 1c: Comprehensive Detection Performance

Overall Metrics across all evaluation streams:

Method	Precision	Recall (EDR)	F1-Score	Delay	False Pos.
DAWIDD	0.435	0.734	**0.532**	38	10.3
IDW_MMD	0.432	0.737	0.531	36	9.6
SE_CDT	0.432	0.737	0.531	36	9.6
ShapeDD_IDW	0.432	0.737	0.531	36	9.6
MMD	0.429	0.734	0.527	35	10.5
ShapeDD	0.377	0.749	0.491	34	13.2
D3	**0.558**	0.472	0.489	**16**	**0.6**
KS	0.224	**0.775**	0.335	34	27.2

- **Synthetic $P(X)$ drifts** (Gaussian, Random Uniform): strong F1 on moderate-to-severe shifts (0.69–0.74), degrading gracefully on mildest case (0.51), with FP ≤ 10.
- **Recurrent/Blip** (Rbfblips, Stagger Recurrent): reliably detected (F1 $\approx$ 0.74–0.76) with minimal FP.
- **Real-drift controls** (SEA, Hyperplane, LED): low F1 *by design* — no $P(X)$ shift is present, confirming no false alarms on pure $P(Y|X)$ drift.

Result 1c: Per-Dataset Detection F1 (1/2)

Part 1: Basic & Gradual/Mild Drift

Method	Electricity	Gaussian Gradual Synthetic	Gaussian Moderate	Random Mild
DAWIDD	0.250	**0.472**	0.753	**0.594**
IDW_MMD	**0.383**	0.470	0.736	0.511
SE_CDT	**0.383**	0.470	0.736	0.511
ShapeDD_IDW	**0.383**	0.470	0.736	0.511
MMD	0.291	0.462	0.740	0.566
ShapeDD	0.267	0.451	0.700	0.529
D3	0.213	0.029	**0.998**	0.012
KS	0.263	0.315	0.356	0.335

Part 2: Random Uniform & Hyperplane

Method	Random Moderate	Random Severe	Random Ultra Severe	Hyperplane
DAWIDD	**0.728**	0.739	0.738	0.136
IDW_MMD	0.688	0.708	0.712	0.179
SE_CDT	0.688	0.708	0.712	0.179
ShapeDD_IDW	0.688	0.708	0.712	0.179
MMD	0.685	0.707	0.709	0.169
ShapeDD	0.619	0.636	0.638	0.197
D3	0.244	**0.931**	**0.933**	**0.000**
KS	0.407	0.434	0.434	0.240

Result 1c: Per-Dataset Detection F1 (2/2)

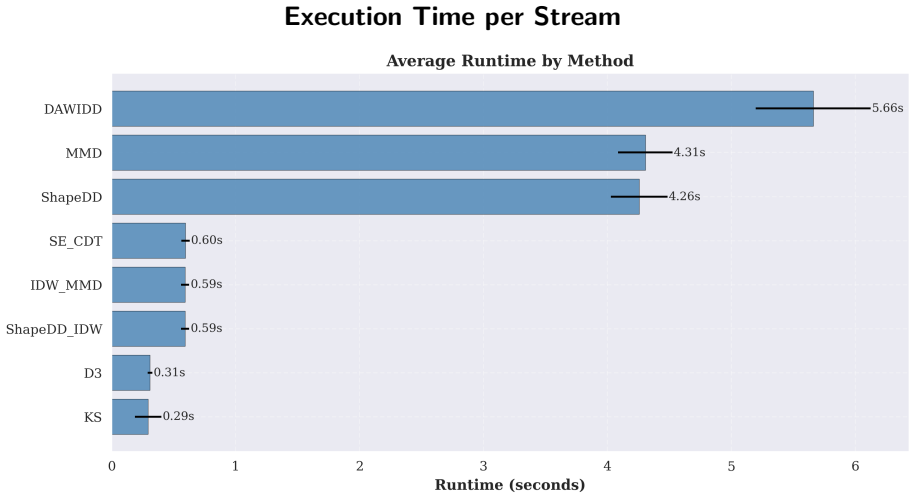
Part 3: Led, Rbfblips & Stagger

Method	Led Abrupt	Rbfblips	Stagger	Stagger Recurrent Explicit
DAWIDD	0.130	0.749	0.750	0.749
IDW_MMD	0.148	0.744	0.755	0.753
SE_CDT	0.148	0.744	0.755	0.753
ShapeDD_IDW	0.148	0.744	0.755	0.753
MMD	0.127	0.747	0.742	0.739
ShapeDD	0.163	0.670	0.681	0.679
D3	**0.000**	**1.000**	**0.998**	**1.000**
KS	0.071	0.371	0.459	0.458

Part 4: Sea & Overall Mean F1

Method	Standard Sea	Mean
DAWIDD	0.129	**0.532**
IDW_MMD	0.118	0.531
SE_CDT	0.118	0.531
ShapeDD_IDW	0.118	0.531
MMD	0.163	0.527
ShapeDD	0.151	0.491
D3	**0.000**	0.489
KS	0.206	0.335

Result 1d: Runtime Efficiency (1/2)



Result 1d: Runtime Efficiency (2/2)

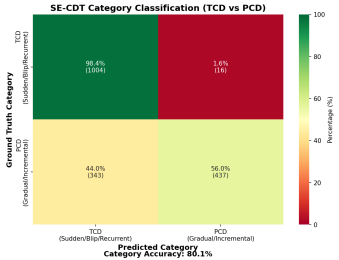
Execution Time per Stream

Method	Runtime (s/stream)	Throughput (samples/s)	Speedup vs ShapeDD
KS	0.29	33,990	14.47×
D3	0.31	32,300	13.75×
ShapeDD-IDW	0.59	16,858	7.18×
IDW-MMD (standalone)	0.59	16,849	7.17×
SE-CDT	0.60	16,764	7.14×
ShapeDD (original)	4.26	2,349	1.00×
MMD	4.31	2,322	0.99×
DAWIDD	5.66	1,767	0.75×

Computational Speedup

Gamma-null approximation makes SE-CDT roughly **7× faster** than permutation-based ShapeDD (**0.60 s** vs. **4.26 s** per 10k samples) and standard MMD (4.31 s), bringing kernel-based detection into the latency class of lightweight tests like KS and D3.

Result 2a: Unsupervised Classification (SE-CDT in Oracle Mode)



Two enhancements added

Concept Memory: ring buffer of post-drift snapshots (Standard-MMD match) → recovers Recurrent typing.
Online Self-Calibration: rolling-quantile baseline adapts SNR/WR/CV thresholds → robustness on noisy streams.

Headline — label-free typing

CAT = 80.1% (TCD vs PCD — the decision that drives adaptation), at **92.6% recall**, with **no labels**.

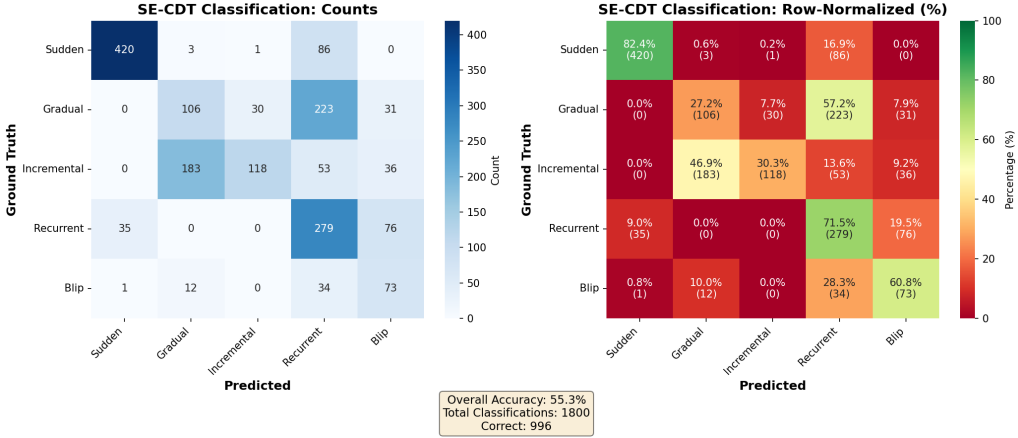
Per-type subtype accuracy (1800 events):

Drift Type	Acc.	Notes
Sudden	**82.4%**	Sharp, well-separated peaks
Recurrent	**71.5%**	Enabled by Concept Memory
Blip	**60.8%**	Multi-peak tolerance rule
Incremental	30.3%	Overlaps Gradual on VR
Gradual	27.2%	57% absorbed into Recurrent

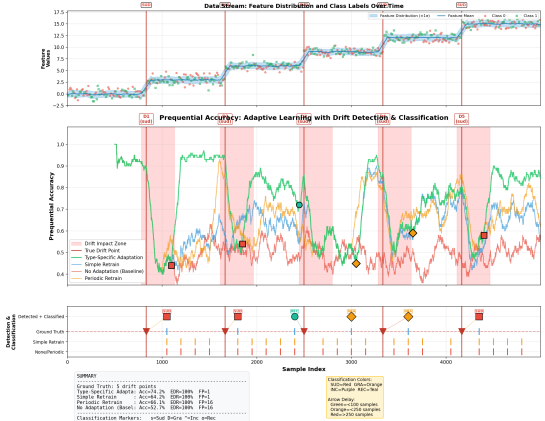
Honest limitation

SUB = 55.3%: slow types overlap on Variance Ratio — the weak point. Supervised CDT-MSW scores higher SUB but needs labels.

Result 2b: Confusion Matrix



Result 3: Model Adaptation (Normal / Streaming Mode)

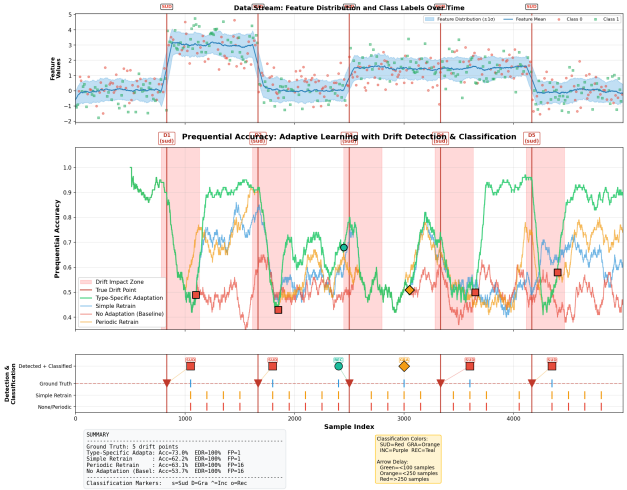


Strategy	Step.	Sudd.
Type-Specific (Green)	**71.4%**	**70.8%**
Simple Retrain (Blue)	63.0%	62.2%
Periodic Retrain (Orange)	64.0%	63.7%
No Adaptation (Red)	54.8%	53.7%

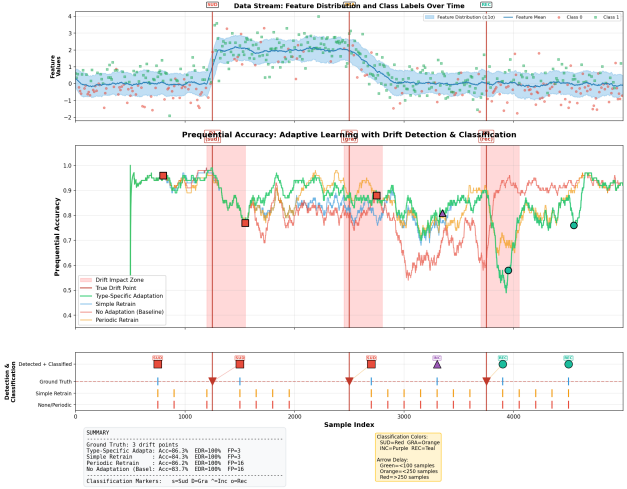
Why Type-Specific helps

- **Avoids Waste:** Resetting on gradual drift discards history; periodic retrains ignore signals.
- **Optimal Update:** Sudden $\rightarrow$ full reset; Incremental $\rightarrow$ warm-start; Gradual $\rightarrow$ recency-weighted retrain.
- **Gains: +16.6 pp** (Stepping) and **+17.1 pp** (Sudden) vs No Adaptation ($p < 10^{-15}$).

Result 3b: Adaptation on Sudden Drift

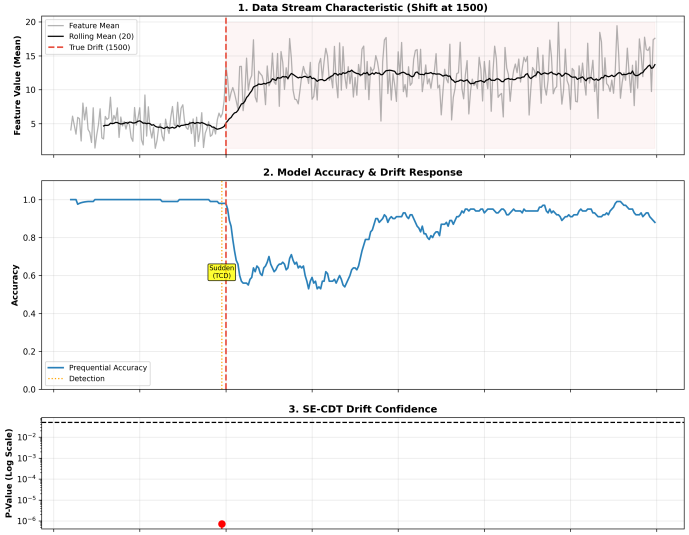


Result 3c: Adaptation on Mixed Drift Scenario



Agenda

End-to-End Deployment Prototype on Apache Kafka



Drift at sample 1500 → accuracy dips → SE-CDT fires ($p \approx 10^{-6}$); CPU/memory stay flat (backup).

Agenda

Contributions of This Thesis

1. **Detection (Layer 1):** A calibrated, label-free detector — ShapeDD with IDW optimal weighting and a **Gamma-null** p-value that replaces the permutation test for speedup.
2. **Classification (Layer 2): SE-CDT** — a fully *unsupervised* drift-type classifier that reads 9 geometric/temporal shape attributes (plus a Variance Ratio) of the MMD trace.
3. **Adaptation (Layer 3):** A **type-aware** routing policy mapping each drift type to its optimal update, validated end-to-end on an Apache Kafka prototype.

Conclusion

Summary

1. **Detection:** ShapeDD-IDW with Gamma-null p-value provides a calibrated, label-free test with an end-to-end speedup.
2. **Classification:** SE-CDT operates label-free, achieving high Category Accuracy (80.1%) and event recall (92.6%).
3. **Adaptation:** Type-Specific routing yields substantial gains (+17.1 pp on Sudden) over no adaptation.
4. **System:** Validated through an end-to-end Kafka prototype deployment.

Key Limitations

- **Subtype Confusion:** Gradual and Incremental shifts remain challenging (27.2% / 30.3% accuracy) due to overlap on the Variance Ratio.
- **Empirical Thresholds:** The Concept Memory threshold ($\mu = 0.15$) was chosen empirically based on the H0 noise floor.
- **Adaptation Evaluation Scope:** Tested heavily on sudden-dominated streams; robustness on purely gradual/incremental scenarios requires further study.

References I

M. Harries, "Splice-2 comparative evaluation: Electricity pricing," 1999.

W. N. Street and Y. Kim, "A streaming ensemble algorithm (sea) for large-scale classification," in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, pp. 377–382.

G. Hulten, L. Spencer, and P. Domingos, "Mining time-changing data streams," in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, pp. 97–106.

J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, ser. Lecture Notes in Computer Science, vol. 3171. Springer Berlin Heidelberg, 2004, pp. 286–295.

M. Baena-García, J. del Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Proceedings of the 4th ECML PKDD International Workshop on Knowledge Discovery from Data Streams (IWKDDS 2006)*, Berlin, Germany, 2006, pp. 77–86.

H. Guo, H. Li, Q. Ren, and W. Wang, "Concept drift type identification based on multi-sliding windows," *Information Sciences*, vol. 585, pp. 1–23, 2022.

Z. Wang and W. Wang, "Concept drift detection based on kolmogorov–smirnov test," in *Artificial Intelligence in China*. Springer Singapore, 2020, pp. 273–280.

References II

F. Hinder, J. Brinkrolf, V. Vaquet, and B. Hammer, "A shape-based method for concept drift detection and signal denoising," in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2021, introduces ShapeDD algorithm for shape-based drift detection.

THANK YOU FOR LISTENING!

Q & A

Student: Le Phuc Duc
Supervisor 1: Assoc.Prof.Dr. Thoai Nam
Supervisor 2: Dr. Nguyen Quang Hung

Backup: SE-CDT Classification Algorithm

Algorithm: SE-CDT — Unsupervised Drift Type Classification

Require: Drift magnitude signal $\sigma(t)$ from Detection Module
Require: Raw data window W for Variance Ratio
Ensure: Drift type: {TCD-Blip, TCD-Sudden, TCD-Recurrent, PCD-Gradual, PCD-Incremental}

$\sigma_s \leftarrow \text{GaussianFilter}(\sigma,\ \sigma_g)$
$P \leftarrow \text{FindPeaks}(\sigma_s,\ \bar{\sigma}_s + \alpha\,\sigma_{\text{std}})$
Compute features: $n_p \leftarrow |P|$, WR, CV, SNR, PPR, DPAR, LTS, SDS, MS
$\text{VR} \leftarrow \text{VarianceRatio}(W)$
$\ell \leftarrow \text{EstimateDriftDuration}(\sigma_s)$
if $n_p \leq \tau_n^{\text{blip}}$ **and** $\text{WR} < \tau_{\text{WR}}^{\text{blip}}$ **and** BlipProfile(...) **then**
    return **TCD-Blip**
end if
if $\ell > \tau_\ell$ **then**
    if IsIncremental($\text{VR}, \text{LTS}, \ldots$) **then**
        return **PCD-Incremental**
    else
        return **PCD-Gradual**
    end if
end if
if $n_p \leq \tau_n^{\text{sud}}$ **and** $\text{WR} < \tau_{\text{WR}}$ **and** $\text{SNR} > \tau_{\text{SNR}}$ **then**
    return **TCD-Sudden**
end if
if $n_p \geq \tau_n^{\text{rec}}$ **and** $\text{CV} < \tau_{\text{CV}}$ **and** $\text{LTS} < \tau_{\text{LTS}}$ **then**
    return **TCD-Recurrent**
end if
if IsIncremental($\text{VR}, \text{LTS}, \ldots$) **then**
    return **PCD-Incremental**
else
    return **PCD-Gradual**
end if

Backup: What the 9 Shape Attributes Mean

All measured on the smoothed MMD signal $\sigma_s(t)$; a peak = a point above $\bar{\sigma}_s + 0.3\,\sigma_{\text{std}}$.

Geometric — peak structure ("what it looks like")

- n_p (peak count): 1 → Sudden; 2–3 → Blip; many → Recurrent.
- **WR** $= \text{FWHM}/(2l_1)$: small → sharp peak (Sudden); large → broad hump (PCD).
- **CV** $= \text{std}/\text{mean}$ of inter-peak gaps: low → evenly spaced (Recurrent).
- **SNR** $= \max/\text{median}$: high → one clean peak above the noise (Sudden).
- **PPR**: gap between 2 tallest peaks / length; small → two close peaks (Blip).
- **DPAR** = min / max of the 2 peak heights; near 1 → twin peaks (Blip).

Temporal — time behaviour ("how it evolves")

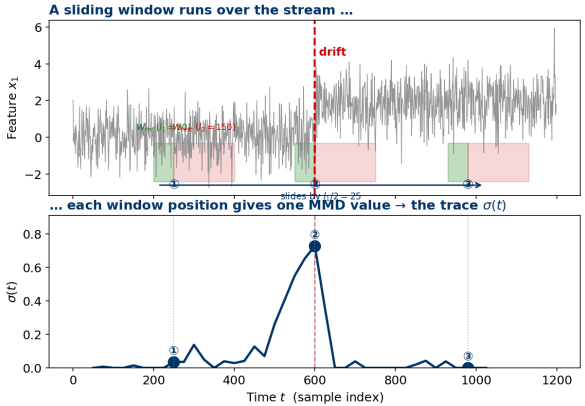
- **LTS**: R^2 of a linear fit; high → straight ramp (Incremental).
- **SDS**: fraction of jumps $> 1.5\,\text{std}$; high → step-like (Incremental).
- **MS** $= |\uparrow - \downarrow|/(\uparrow + \downarrow)$: high → one-directional (Incremental); low → oscillating (Gradual).

Quick guide

Sudden vs slow → **WR, SNR**; Blip → **PPR+DPAR**; Recurrent → n_p**+CV**; Incr. vs Grad. → **LTS/SDS/MS** (then **VR**).

The 6 geometric attributes answer *what the peak looks like*; the 3 temporal ones answer *how it evolves* — together they separate the 5 drift types with no labels.

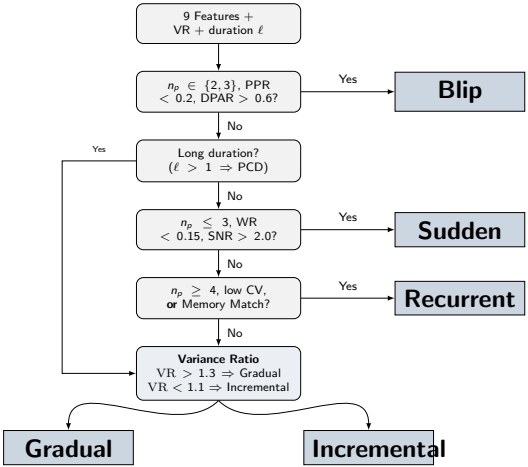
Backup: How the Sliding Window Builds the MMD Trace



- Window = W_{ref} (l_1=50) + W_{test} (l_2=150), slides by $l_1/2 = 25$.
- **(1)** before drift → low; **(2)** straddling → peak; **(3)** after → low again.
- Held in a **ring buffer** (≈750), re-checked every ∼150 samples.
- On a confirmed drift: read the **trace shape** (9 attributes) + ±**750 raw** (Variance Ratio).

Backup: SE-CDT Decision Tree (full)

Rule-based Logic to Identify Drift Types



Backup: IDW-MMD vs Optimally-Weighted MMD (Bharti et al.)

Question: "How is IDW-MMD different from the Optimally-Weighted MMD of Bharti et al. (2023)?"

Bharti et al. (OW-MMD)

- Domain: **likelihood-free inference** (ABC).
- Weights chosen by optimising against a target posterior.
- Requires solving a constrained convex optimisation per call.
- Not designed for streaming drift detection.

IDW-MMD (this work)

- Domain: **drift detection** in fast data streams.
- Weights from a closed-form heuristic: $w_i \propto 1/(\sqrt{d_i} + \epsilon)$.
- Cost: $O(n^2)$ for the kernel matrix, $O(n)$ extra for the weights — no optimisation step.
- Same *weighted MMD* family, different objective and very different runtime.

Summary: both are weighted MMD variants, but they target different problems (ABC vs streaming drift), use different weight functions, and have very different runtime profiles.

Backup: "F1 close to MMD baseline — where is the contribution?"

Question: "SE-CDT's F1 (0.531) is only marginally above MMD (0.524). Where is the gain?"

F1 alone hides the trade-off:

	SE-CDT	MMD	KS
F1	0.531	0.524	0.335
FP	**9.6**	10.6	27.2
Recall	0.737	0.732	0.775
Precision	0.432	0.426	0.224

In a production setting:

- Frequent FPs → **alert fatigue**, lost trust in the monitor.
- Investigating one false alarm typically costs more than missing one event.
- A miscalibrated permutation-style test inflates either FPs or misses.

What SE-CDT actually adds:

1. **Single-test calibrated near** $\alpha = 0.05$ — Standard MMD / IDW-MMD Type-I error 0.025–0.075; the SE-CDT composite is anti-conservative (0.080–0.140), reported honestly as a pipeline limitation.
2. **Classification module on top of detection** — gives a drift type, not just a flag.
3. **Modest runtime speedup over ShapeDD** thanks to the Gamma-null p-value (no kernel re-build per shuffle).
4. Stays **fully unsupervised** at runtime.

Backup: Why does Concept Memory matter for Recurrent drift?

Question: "Recurrent drift requires concept matching — how does Concept Memory provide it?"

The detection challenge:

- A Recurrent stream is a sequence of *repeated* concept switches; each switch produces an MMD peak that, in isolation, looks like a Sudden drift.
- Without memory of past concepts, repeated visits are scored independently → Recurrent collapses into the Sudden branch.

How Concept Memory resolves it:

1. During every *stable period*, take a snapshot of the current data window and store it in a ring buffer of size $K = 8$.
2. When a new drift fires, compute Standard MMD between the new post-drift window and every snapshot.
3. If the smallest MMD $< \mu = 0.15$ → override the

Why $\mu = 0.15$?

$\mu = 0.05$ falls inside the MMD noise floor (false matches between independent stationary windows). An empirical sweep on H0 calibration data shows $\mu = 0.15$ separates "revisited concept" from "new concept" reliably.

Remaining limitations

Two failure modes remain on the harder Recurrent variants:

- Recurrent events that are *too far apart* push the original snapshot out of the ring buffer.
- Repeated Gradual streams saturate after a few events, so the post-drift window matches a stored snapshot and is labelled Recurrent

Backup: "CDT-MSW has higher CAT and SUB — is SE-CDT worse?"

Question: "CDT-MSW reports CAT 87.0% and SUB 86.7%, both higher than SE-CDT. Why frame SE-CDT as competitive?"

Side-by-side numbers

	SE-CDT	CDT-MSW
Labels	**None**	Required
CAT	80.1%	87.0%
SUB	55.3%	86.7%
Recall	**92.6%**	82.5%
Effective	**74.2%**	71.8%

Effective = CAT × Recall (probability of seeing a drift *and* typing it correctly). CDT-MSW Recall = $1 - \overline{\text{MDR}}$ from paper Table 5.

Reading the numbers in context:

- CDT-MSW reports CAT and SUB *conditionally* on drifts it detects; with paper recall $\approx 82.5\%$ it still misses $\sim 17\%$ of events.
- SE-CDT detects 92.6% of drifts and types every one of them without any labels.
- End-to-end, SE-CDT lands 2.4pp above CDT-MSW on effective accuracy (74.2% vs 71.8%); the contribution is reaching this on **unsupervised** $P(X)$.

Trade-off

CDT-MSW is more accurate per detected event because it has access to ground truth labels. SE-CDT

Backup: "Why not deep learning?"

Question: "Couldn't a deep learning model solve this more accurately?"

Deep models for drift detection — limitations

- Need a large labelled dataset to train — the very thing we don't have on a streaming pipeline.
- Higher inference latency per window than a kernel computation.
- No closed-form theoretical bound on the test statistic.
- Decision is hard to explain to a domain expert.

Why MMD / ShapeDD fits this problem

- Fully unsupervised — no training run before deployment.
- Theoretical guarantees: Triangle Shape Theorem, Type-I error from H0 calibration.
- Decision rule is human-readable (a small tree of geometric features).
- One mechanism works across $P(X)$ shifts of arbitrary type.

Future direction: keep the kernel framework but plug in a learned feature extractor for high-dimensional inputs (Chapter 6).

Backup: behaviour on mild drift

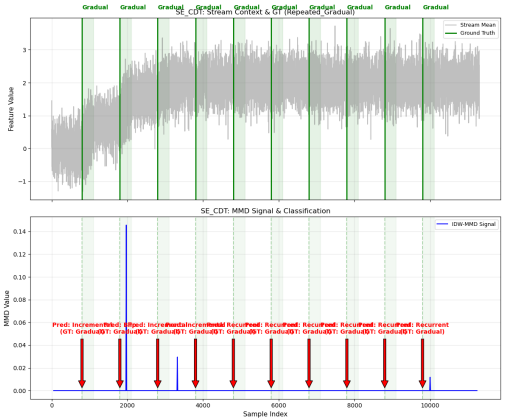
Question: "On the mild-drift scenario (Random Mild, $\delta = 0.125$) SE-CDT scored F1 $= 0.511$ vs MMD $= 0.563$ — where does the gap come from?"

Cause:

- IDW down-weights points in dense (stable) regions.
- Mild drift lives mostly inside those regions, shortening the IDW peak.
- Net effect: some peaks fall under threshold, reducing recall.

This is the intended trade-off:

- IDW-MMD biases toward fewer false alarms (FP ~ 9.6/run).
- Cost: $\sim$ 5pp F1 drop on the smallest shifts.



Possible mitigations (future work)

Backup: Permutation test vs Gamma-null p-value

Question: "Removing the permutation test — is the Gamma-null p-value still trustworthy?"

Permutation test (original ShapeDD)

- Reshuffles indices many times per window.
- Empirical null, no distributional assumption.
- Each shuffle re-builds the kernel matrix $\rightarrow$ heavy compute.

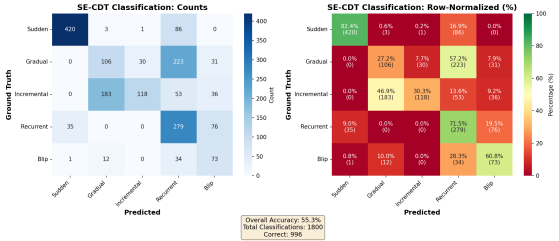
Gamma-null p-value (IDW-MMD)

- Moment-matched Gamma fit to the null MMD distribution (Gretton et al. 2009).
- Estimates the first two moments from a small number of index shuffles, reusing the kernel matrix.
- Complexity $O(n^2)$ for the kernel plus a few cheap shuffles.

Verification: The detection F_1 remains comparable to the permutation baseline; the H0-calibration on three stationary distributions reports Standard MMD and IDW-MMD inside the 95% CI of $\alpha = 0.05$ (0.025–0.075), while the SE-CDT composite is anti-conservative at 0.080–0.140 (up to $\approx 2.8 \times \alpha$), reflecting peak-selection and sliding-window dependence rather than the per-candidate test itself. End-to-end runtime is slightly faster than the ShapeDD baseline.

Backup: “Subcategory accuracy 55.3% — is it close to random?”

Question: “With 5 classes, is $\sim 60\%$ close to random guessing?”



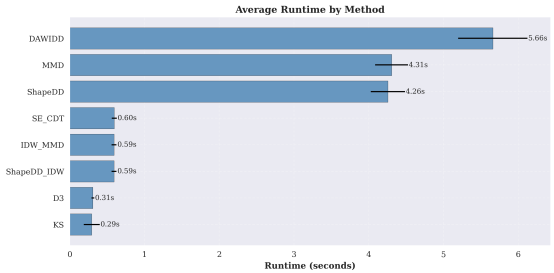
It is well above random:

- Random baseline across 5 classes is 20%.
- Micro SUB $= 55.3\%$ is $\sim 2.8\times$ random and well above the 95% CI of a random predictor ($\sim 23\%$).
- Performance is **very uneven across classes**:

Drift Type	Accuracy
Sudden	**82.4%**
Incremental	30.3%
Recurrent	**71.5%**

Backup: Computational Overhead & Runtime Analysis

Question: "What is the computational overhead of IDW-MMD compared to standard methods?"



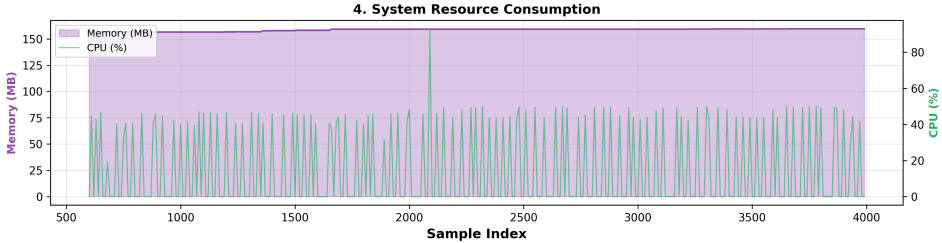
Complexity & Efficiency:

- **Standard MMD / IDW-MMD:** $O(n^2)$ matrix computation, but IDW computes weights once per window step.
- **Gamma-null Approximation** bypasses the permutation test entirely, running in constant extra shuffles.
- **ShapeDD-IDW** is roughly **7× faster** than the original permutation-based ShapeDD (0.59s vs 4.26s per 10k samples).

Streaming Suitability

In online deployments, the detector easily processes streams at over **16k samples/sec**, making its latency negligible in streaming Kafka pipelines.

Backup: Kafka Deployment — System Resource Usage



Memory and CPU stay stable throughout the run — SE-CDT's streaming overhead is negligible.

Backup: Why are Gradual (27.2%) and Incremental (30.3%) low?

Question: "The two slow types are the weak point — what exactly goes wrong?"

Two systematic confusions:

1. **Gradual ↔ Incremental overlap:** Both land in ambiguous Variance Ratio bands (∼47% of Incremental read as Gradual).
2. **Saturation → Recurrent:** In repeated slow drifts, $P(X)$ stops changing. The post-drift window matches stored snapshots, labelling it Recurrent (∼57% of Gradual events).

Why CAT is unaffected

Both confusions stay *inside* PCD, preserving TCD/PCD accuracy (CAT = 80.1%). Confusing Gradual with Incremental is low-cost for adaptation.

Fix (future work)

Replace fixed VR thresholds with learned classifiers and separate Concept-Memory match step from shape classification.

Backup: "Is the adaptation gain Sudden-biased?"

Question: "Type-Specific wins +17pp — but the classifier is only ∼55% on subtypes. Does the gain depend on the easy scenarios?"

Honest answer — yes, partly:

- The three adaptation scenarios (Stepping, Sudden, Mixed) are all **Sudden-dominated**, exactly where the classifier is strongest (Sudden 82.4%).
- So routing is correct *most of the time in this region*, and any reset on a true sudden drift helps regardless.
- We have **not** tested adaptation on Gradual/Incremental-heavy streams, where subtype accuracy drops to 27–30%.

Why the gain still holds

The category decision (TCD vs PCD, 80.1%) drives the adaptation policy, not the exact subtype. TCD→reset and PCD→fine-tune is the right call even when the subtype within PCD is wrong — the operational cost of confusing Gradual with Incremental is small (same update family).

Scope claim

We claim the adaptation benefit *for the sudden/recurrent regime*, not universally. Gradual/Incremental-heavy adaptation is listed as future work (Ch. 6).